

Building Conversers with vivid-creator

A Framework for Domain-Specific Conversational Agents

What is vivid-creator?

An 8-step guided methodology for building conversers:

- Domain-specific conversational agents
- Deep expertise in one subject
- Consistent voice via CSV training
- Quality dialogue over task execution

Your role: Guide developers through the process by reading instruction files and applying them to their domain

Example: Movivid

A movie converser that:

- Asks about user's life situation
- Recommends films based on emotions & philosophy
- Speaks in first-person ("I've spent time with this film...")
- Uses 30 curated philosophical films

Voice: Warm, empathetic, playful

Step 1: System Prompt Setup

Define the converser's identity

- Domain expertise (movies, music, books...)
- Conversational style (playful, professional, poetic)
- Primary purpose (entertainment, education, support)
- Interaction pattern (ask about user first? or respond to queries?)
- Boundaries (what's in/out of scope)

Output: Initial system prompt draft

Movivid Example

Domain: Movies (philosophical & emotional depth)

Style: First-person, warm, playful

- "I've spent time with these films..."
- "What sticks out for me is..."

Purpose: Help users find films for their life situations

Pattern: Asks about user's life first, then recommends

Boundaries: Focus on themes, not technical film analysis

Step 2: Data API Discovery

Understand what data is available

1. Identify APIs for the domain
2. Test actual responses (don't assume!)
3. Create API wrapper classes
4. Document capabilities & gaps

Critical: Inspect real JSON responses before building

Output: Working API wrappers + documentation

Movivid Example

APIs chosen:

- **OMDb**: IMDb ratings, plot, cast
- **uNoGS**: Netflix availability

Wrappers created:

- `omdb_wrapper.py` with factory function
- `unogs_wrapper.py` for catalog search

Gaps identified:

-  No philosophical analysis
-  No personal reflections
- → Need essays in curated catalog

Step 3: InternetToText Functions

Fill knowledge gaps - TWO approaches:

APPROACH 1: PREPROCESSING (what Movivid used)

- Build database BEFORE converser runs
- Research → Categorise → Summarise → Save
- Review/edit before use
- **When:** Quality critical, curated domain

APPROACH 2: DYNAMIC (runtime)

- Fetch DURING conversation
- User query → Search → Respond
- **When:** Need real-time data, unpredictable queries

Movivid Example

Chose **PREPROCESSING**

Problem: APIs lack philosophical analysis

Solution:

- `film_theme_analyzer.py` script
- Research → Extract themes → Generate essays
- Create 2 essays per film (4000 chars each):
 - `philosophical_context`
 - `personal_reflection`
- Save to catalog

Result: 30 films with curated, reviewed essays

Pattern templates:

Step 4: Function Breakdown

Design 6-12 functions across 4 types

1. **CHANGE_CONTEXT** (2-4): Update state
2. **WORDALISE** (3-6): Generate responses with CSV training
3. **INSTRUCTIONS** (1-2): Explain how converser works
4. **ENGAGE** (1-2): Prompt user to share

Each function:

- Has clear purpose
- Defined parameters
- Schema for function calling

Note: Do this BEFORE creating CSV examples in Step 5

Movivid Example

5 functions total:

1. `changeFilm` (CHANGE_CONTEXT)
 - Updates `current_entities`
2. `filmsBiggerMeaning` (WORDALISE)
 - Philosophical themes with CSV training
3. `whatFilmSaidToMe` (WORDALISE)
 - Personal reflections with CSV training
4. `howMovividWorks` (INSTRUCTIONS)
 - LLM-generated greeting
5. `tellMeAboutYou` (ENGAGE)

Step 5: Wordalisations Training (ESSENTIAL)

CSV-based few-shot training

Format: `question;knowledge;answer`

Process (for each WORDALISE function from Step 4):

1. Take first WORDALISE function
2. Create CSV file
3. Write 3-5 hand-crafted examples
4. Generate 5-10 more with script
5. Test and refine
6. Move to next WORDALISE function

Goal: 10-15 high-quality examples per function

This is THE way conversers learn voice

Note: Steps 4-8 often iterate

Movivid Example

Two CSV files (for 2 WORDALISE functions):

- `filmsBiggerMeaning.csv` (3 examples)
- `whatFilmSaidToMe.csv` (2 examples)

Example from whatFilmSaidToMe.csv:

```
question;knowledge;answer
What did Eternal Sunshine say to you?;"[4000 char essay]";"Eternal Sunshine taught me that memory isn't just a record—it's how we love..."
```

Script: `train_wordalisations.py` generates more examples using GPT + existing examples as few-shot

Step 6: Implementation

Build the converser class

- Inherit from `Converser` base class
- Implement each function type:
 - WORDALISE: ONE LLM call (system + history + few-shot)
 - CHANGE_CONTEXT: Update state only
 - INSTRUCTIONS/ENGAGE: Return content
- Set up Azure OpenAI integration
- Track context (entities, user_profile, history)

Output: Working converser class

Movivid Example

File: `movivid.py`

Key components:

```
class Movivid(Converser):
    def __init__(self, catalog_path, csv_dir):
        self.catalog = load_catalog()
        self.csv_db = WordalisionsCSV(csv_dir)

    @property
    def instructions(self):
        # Routing logic (Step 7)
        return "If user asks..."

    def _initialize_functions(self):
        return [ChangeFilm(...),
                FilmsBiggerMeaning(...), ...]
```


Step 7: Routing Logic

Write IF-THEN routing rules

Define `instructions` property with:

- When to call each function
- How to handle ambiguity
- Parallel calling patterns
- Edge cases

Pattern: "If user asks X, call function Y"

This goes in the system prompt for Azure
OpenAI function calling

Movivid Example

Routing logic in `instructions` property:

```
"If user says hello or asks what this is:  
  → Call howMovividWorks  
  
If user asks 'Tell me about [film]':  
  → Call changeFilm  
  
If user asks about philosophical themes:  
  → Call filmsBiggerMeaning  
  
If user asks 'what did it say to you?':  
  → Call whatFilmSaidToMe  
  
If user shares life situation:  
  → Call tellMeAboutYou"
```

Step 8: Testing & Refinement

Validate and iterate

1. Create test conversation script
2. Run conversations
3. Check:
 - Are functions called correctly?
 - Is voice consistent?
 - Are responses on-brand?
4. Refine:
 - Edit CSV examples if voice is off
 - Update routing if functions aren't triggered
 - Test again

Iterate until converser works well

Movivid Example

Test script: `test_conversation.py`

Test conversation:

1. "Hi! What is this?" → `howMovividWorks` ✓
2. "Tell me about The Matrix" → `changeFilm` ✓
3. "What's the bigger meaning?" → `filmsBiggerMeaning` ✓
4. "What did it say to you?" → `whatFilmSaidToMe` ✓

Refinements made:

- Shortened CSV examples (were too long)
- Fixed prompt duplication issue
- Removed second LLM call wrapping

Key Architecture: WORDALISE Functions

ONE LLM call generates final response

Prompt structure:

1. System prompt (converser identity)
2. Conversation history (last 4-6 messages)
3. Few-shot prompt:
 - 3-5 CSV examples
 - Domain knowledge (film essays)
 - Current question

No second "wrapping" call

Result: Response in converser's voice

Movivid Example

For **filmsBiggerMeaning** :

System: "You are Movivid, a warm film converser..."

History:

User: "Tell me about The Matrix"

Assistant: "Great choice! Let me explain..."

Few-shot prompt:

[3 CSV examples showing Movivid's voice]

[Philosophical essay about The Matrix]

Question: "What's the bigger meaning?"

→ GPT generates answer in Movivid's voice

→ Returned directly to user

Result: Movivid is Self-Contained

Everything in one folder:

```
examples/movivid/  
├── movivid.py (main class)  
├── movivid_functions*.py  
├── *_wrapper.py (APIs)  
├── wordalisations/ (CSV training)  
├── data/ (film catalog)  
├── scripts/ (training tools)  
└── tests/ (test conversation)
```

Standalone: No framework dependency needed to improve it

Developers can:

- ✓ Add training examples manually
- ✓ Generate examples with script
- ✓ Test conversations
- ✓ Add more films to catalog
- ✓ Create new functions
- ✓ Refine voice independently

All tools included:

- TRAINING_EXAMPLES.md guide
- train_wordalisations.py script
- test_conversation.py tester
- Complete documentation

Summary: The vivid-creator Process

8 Steps to Building a Converser

1. **System Prompt** - Define persona
2. **Data APIs** - Test and wrap external data
3. **InternetToText** - Fill knowledge gaps (optional)
4. **Function Breakdown** - Design 6-12 functions (4 types)
5. **Wordalisations** - Create CSV training examples ★
6. **Implementation** - Build converser class
7. **Routing Logic** - Write IF-THEN rules
8. **Testing** - Validate and refine

Key insight: Function design (Step 4) comes BEFORE training examples (Step 5). You need to know what functions exist before creating CSV examples for them.

Iteration: Steps 4-8 often iterate, but function breakdown comes first.

vivid-creator's role: Read instruction files, guide developer through each step, apply methodology to their

Next Steps

To use vivid-creator:

1. Invoke "vivid-creator" mode in your AI assistant
2. Describe your converser domain
3. Follow the 8-step guided process

To study the example:

```
cd examples/movivid  
python tests/test_conversation.py
```

To read the methodology:

```
.github/instructions/building_conversers_overview.instructions.md
```

Framework: `framework/converser.py` - Base classes for all conversers

Questions?

Documentation:

- Complete methodology: `.github/instructions/building_conversers_*.instructions.md`
- Movivid case study: `.github/instructions/movivid.instructions.md`
- Training guide: `examples/movivid/TRAINING_EXAMPLES.md`

Key insight: CSV training is how conversers maintain consistent voice. Quality examples > quantity.

Philosophy: Learn by building. The methodology evolves through practice.

